

Python 04 Conditionals: True or False?

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM14.html

Quick links:
Click on any [↑](#) to get back here!

"Simple" conditions

- membership `in`
- `try except` for checking code will work
- many different types of True/False conditions
- `if` statements
- `for ... if`
- nested `if` statements
- `else` statements
- `elif` statements
- using `break`
- using a `while` loop
- `append` and `pop` a list

Complex conditions

- combining conditionals: `and`
- combining conditionals: `or`
- splitting code up to "make it look nice"

Exercises

1. [processing tabular data](#)
2. [Kmer counting](#)
3. [pairwise distance](#)

True or False

Conditions are things that can be evaluated as being either `True` or `False`.

A simple (incorrect!) example comparing two numbers

python3 # or p3

Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.

4 = 5

```
File "<stdin>", line 1
SyntaxError: can't assign to literal
```

A simple (correct this time) example comparing two numbers

4 == 5

False

4 >= 5

False

4 <= 5

True

len("ATGC") > 5

False

"GAATTC".count("T") > 1

True

"ATGCTT".startswith("ATG")

True

"ATGCTT".endswith("TTT")

False

"ATGCTT".isupper()

Yes

"ATGCTT".islower()

No



B in BAT

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'B' is not defined
```

"B" in "BAT"

True

"B" in ["B","A","T"]

True

"Q" in "BAT"

False

It can't deal with all True/False situations though...!

The COP29 climate change conference in Azerbaijan should have been important.

```
File "<stdin>", line 1
  The COP29 climate change conference in Azerbaijan should have been important.
    ^
SyntaxError: invalid syntax
```

Link to [COP29](#)

↑ `try except` for error trapping, the nice way

If you think some part of your code might not work at some particular point, but you don't want the entire script to crash/fail, we can get Python to `try` things first, as described on <https://pythonbasics.org/try-except/>.

An example that should fail (as `x` and `y` dont exist)

```
try :  
    if x > y :  
        "{} is bigger than {}".format(x,y)  
except :  
    f"Something not right!"
```

'Something not right!'

x

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'x' is not defined
```

y

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'y' is not defined
```

x,y=12,9

x

12

y

9

```
try :  
    if x > y :  
        "{} is bigger than {}".format(x,y)  
except :  
    f"Something not right!"
```

12 is bigger than 9

x,y=9,12

x

9

y

12

Run it again: all works, no output, as no error

```
try :  
    if x > y :  
        "{} is bigger than {}".format(x,y)  
except :  
    f"Something not right!"
```

x,y=9,"12"

x

9

y

'12'

Run it again: all works, with output

```
try :  
    if x > y :  
        "{} is bigger than {}".format(x,y)  
except :  
    f"Something not right!"
```

'Something not right!'

We can add other things here too!

Delete the variable x, see what happens!

del x

x

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
NameError: name 'x' is not defined
```

y

'12'

```
try :  
    if x > y :
```

```
    "{} is bigger than {}".format(x,y)
except NameError:
    f"One of the variables wasnt defined it seems..."
except :
    f"Something not right!"
finally :
    f"Done using the try:except test"

'One of the variables wasnt defined it seems...'
'Done using the try:except test'
```

x=3

```
try :
    if x > y :
        "{} is bigger than {}".format(x,y)
except NameError:
    f"One of the variables wasnt defined it seems..."
except :
    f"Something not right!"
finally :
    f"Done using the try:except test"

'Something not right!'
'Done using the try:except test'
```

```
try :
    if x > y :
        "{} is bigger than {}".format(x,y)
except NameError:
    f"One of the variables wasnt defined it seems..."
except TypeError:
    f"One of the variables wasnt the right data type..."
except :
    f"Something not right!"
finally :
    f"Done using the try:except test"

'One of the variables wasnt the right data type...'
'Done using the try:except test'
```

More than one thing wrong: fails on the first one it finds

del x

```
try :
    if x > y :
        "{} is bigger than {}".format(x,y)
except NameError:
    f"One of the variables wasnt defined it seems..."
except TypeError:
    f"One of the variables wasnt the right data type..."
except :
    f"Something not right!"
finally :
    f"Done using the try:except test"

'One of the variables wasnt defined it seems...'
'Done using the try:except test'
```

🏠Types of True/False conditions in Python...

- is equal to ==
- is not equal to !=
- numerical comparisons (>, <, >=, <=)
- string methods (startswith, isupper)
- is a value in a list

These are called operators, and there are many different kinds, some of which we have seen before:

Arithmetic

Operator	Meaning	Example
+	Add two operands or unary plus	$x + y$ +2
-	Subtract right operand from the left or unary minus	$x - y$ -2
*	Multiply two operands	$x * y$
/	Divide left operand by the right one (always results in a float)	x / y
%	Modulus - remainder of the division of left operand by the right	$x \% y$ 7%3 1

//

Floor division - division that results into whole number adjusted to the left in the number line

55/3.3
16.666666666666668
55//3.3
16.0
int(55//3.3)
16

ceiling division
version?!

**

Exponent - left operand raised to the power of right

2.06**3
8.741816

Assignment

Operator	Example	Equivalent to
=	x = 5	x = 5
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5
//=	x //= 5	x = x // 5
**=	x **= 5	x = x ** 5

Comparison

Operator	Meaning	Example
>	Greater than - True if left operand is greater than the right	x > y

<	Less than - True if left operand is less than the right	$x < y$
==	Equal to - True if both operands are equal	$x == y$
!=	Not equal to - True if operands are not equal	$x != y$
>=	Greater than or equal to - True if left operand is greater than or equal to the right	$x >= y$
<=	Less than or equal to - True if left operand is less than or equal to the right	$x <= y$

Logical

Operator	Meaning	Example
and	True if both the operands are True	$x \text{ and } y$
or	True if either of the operands is True	$x \text{ or } y$
not	True if operand is false (complements the operand)	$\text{not } x$

Identity

Operator	Meaning	Example
is	True if the operands are identical (refer to the same object)	$x \text{ is True}$
is not	True if the operands are not identical (do not refer to the same object)	$x \text{ is not True}$

Membership

Operator	Meaning	Example
in	True if value/variable is found in the sequence	5 in x
not in	True if value/variable is not found in the sequence	5 not in x

Bitwise Bitwise operators act on operands as if they were a string of binary digits. It operates bit by bit, hence the name!

- <https://wiki.python.org/moin/BitwiseOperators>
- https://en.wikipedia.org/wiki/Two's_complement

For this table, let's say $x = 10$ (0000 1010 in binary) and $y = 4$ (0000 0100 in binary)

Bitwise		
Operator	Meaning	Example
&	Bitwise AND	Sets each bit to 1 if both bits are 1 $x \& y = 0$ (0000 0000)
	Bitwise OR	Sets each bit to 1 if one of two bits is 1 $x y = 14$ (0000 1110)
~	Bitwise NOT	Inverts all the bits $\sim x = -11$ (1111 0101)
^	Bitwise XOR	Sets each bit to 1 if only one of two bits is 1 $x \wedge y = 14$ (0000 1110)
>>	Bitwise right shift	Shift right by pushing copies of the leftmost bit in from the left, and let the rightmost bits fall off $x >> 2 = 2$ (0000 0010)

<< Bitwise left Shift left by pushing zeros in from
shift the right and let the leftmost bits
 fall off
 $x \ll 2 = 40$ (0010 1000)

📌 Using conditionals: `if` statements

Meanwhile, back to conditionals....!

The simplest thing we can do with a condition is: execute some code if it's True, ignore it if it is False.

It's like a flow-chart of decision-making, or input->process->output, or....
Software available to draw them: [Biorender](#) or [yED](#) (other software is available!).

"Pseudocode"

```
is it a nice day?  
  yes: let's go to the beach
```

Define a numerical variable
expression_level = 125
A simple conditional
<pre>if expression_level > 100 : print("This gene is highly expressed")</pre>
This gene is highly expressed

Important things

- the condition line starts with `if`
- the thing we want to test goes after the if
- the line ends with a colon `:`
- just like for loops, the body is indented
- can have multiple lines in the body

📌 Combining `for` and `if`

"Pseudocode"

```
for each day of the week
    is it a nice day?
    yes: let's go to the beach
```

As we saw in the previous lecture's exercises, we frequently use the `for` loop and `if` conditional together.

Using a `for` loop with an `if` inside it

Set up a list of string items

and then output those that have the letter b in them

Below, `acc` takes the value of each element of the list `accessions`

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93']
for acc in accessions :
    if acc.count("b") > 0 :
        print(acc + " contains letter b")
```

```
ab56 contains letter b
bh84 contains letter b
```

Using a `for` loop with an `if` inside it

with another `if` inside that

Set up a list of string items

and then output those that have the letter b and number 4 in them

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93']
for acc in accessions :
    if acc.count("b") > 0 :
        acc_b = acc
        print(acc + " contains letter b")
        if acc_b.count("4") > 0 :
            print(acc + " contains letter b as well as the number 4")
```

```
ab56 contains letter b
bh84 contains letter b
```

bh84 contains letter b as well as the number 4

What if we wanted to count any instances of b or B ?

```
if acc.count("b") > 0 :
```

could be changed to:

```
if acc.lower().count("b") > 0 :
```

...anywhere else need changing...?

📌 Indentation is absolutely critical!

Code with loops and/or conditions often involves multiple levels of indentation.

"Pseudocode"

```
for each day of the week
    is it a nice day?
        yes: is it the weekend?
            yes: let's go to the beach
```

Watch out for IndentationErrors and indentation mistakes: it matters where the indentations are!

Using a for loop with an if inside it

with another if inside that

<u>Incorrect</u> indenting example 1
--

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93']
for acc in accessions :
    if acc.count("b") > 0 :
        acc_b = acc
        print(acc + " contains letter b")
        if acc_b.count("4") > 0 :
            print(acc_b + " has a 4 in it")
        print("Still working on " + acc + "....")
        print(acc + " contains letter b as well as the number 4")
```

Code is good code: no errors...

```
ab56 contains letter b
Still working on ab56....
ab56 contains letter b as well as the number 4
bh84 contains letter b
bh84 has a 4 in it
Still working on bh84....
bh84 contains letter b as well as the number 4
```

Output says ab56 has a 4 in it!? No it doesnt!
--

Using a for loop with an if inside it

with another if inside that

Incorrect indenting example 2

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93']
for acc in accessions :
    if acc.count("b") > 0 :
        acc_b = acc
        print(acc + " contains letter b")
        if acc_b.count("4") > 0 :
            print(acc_b + " has a 4 in it")
            print("Still working on " + acc + "....")
        print(acc + " contains letter b as well as the number 4")
```

This output is even more incorrect (but the code is good code, it has the right syntax)!

```
ab56 contains letter b
Still working on ab56....
ab56 contains letter b as well as the number 4
bh84 contains letter b
bh84 has a 4 in it
Still working on bh84....
bh84 contains letter b as well as the number 4
hv76 contains letter b as well as the number 4
ay93 contains letter b as well as the number 4
```

↑ `else` statements

The previous `if` examples result in either True (yes) or False (no) answers, and so we either execute the bit of code within the `if` block, or we move onto the next part of the process.

Sometimes we might want to have more options: either

- we execute a bit of code, or
- we execute a different bit of code,

depending on the answers we got from the `if` statement evaluation.

"Pseudocode"

```
for each day of the week
    is it a nice day?
        yes: is it the weekend?
            yes: let's go to the beach
            else no: I'm supposed to be going to my lectures, best go...
        else no: do something at home
```

Decision point with two options

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72', 'cp12']
for acc in accessions :
    if acc.startswith('a') :
        print(acc + " starts with a")
    else :
        print(acc + " doesn't start with a")
```

```
ab56 starts with a
bh84 doesn't start with a
hv76 doesn't start with a
ay93 starts with a
ap97 starts with a
bd72 doesn't start with a
cp12 doesn't start with a
```

The `else` line has nothing on it other than the colon `:`

the `else` block is **indented** and is run when the `if` condition is **False**.

Start of minor detour!

We could use the python notation `if not`, which obviously, can be True or False!
See [how to use if not...](#)

```
for acc in accessions :  
    if not acc :  
        f'No element'  
    else :  
        'Processing list element {}'.format(acc)
```

```
'Processing list element ab56'  
'Processing list element eh84'  
'Processing list element hv76'  
'Processing list element ay93'  
'Processing list element zp97'  
'Processing list element cd72'
```

End of minor detour!

↑ elif statements

**# Sometimes we might want to ask multiple questions
so we could "nest" the conditional statements**

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72', 'cp12']
for acc in accessions :
    if acc.startswith('a') :
        print(acc + " starts with a")
    else :
        if acc.startswith('b' ) :
            print(acc + " starts with b")
        else :
            print(acc + " starts with something else")
```

```
ab56 starts with a
bh84 starts with b
hv76 starts with something else
ay93 starts with a
ap97 starts with a
bd72 starts with b
cp12 starts with something else
```

But each extra option needs an extra level of indentation, which gets a bit unwieldy/error prone if it gets bigger than ~3-5 levels.

Can be better to use **elif** (**el** **se** **if**)

Multiple questions being asked, using elif

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72', 'cp12']
for acc in accessions :
    if acc.startswith('a') :
        print(acc + " starts with a")
    elif acc.startswith('b') :
        print(acc + " starts with b")
    elif acc.startswith('h') :
        print(acc + " starts with h")
    else :
        print(acc + " starts with something else")
```

```
ab56 starts with a
bh84 starts with b
hv76 starts with h
ay93 starts with a
ap97 starts with a
```

```
bd72 starts with b  
cp12 starts with something else
```

Sometimes it's just easiest to pull out the ones we want

Much easier to read, and we can add as many conditions as we like

```
accessions = ['ab56', 'eh84', 'hv76', 'ay93', 'zp97', 'cd72']
```

```
for acc in accessions :  
    if acc.startswith('a') :  
        print(acc + " starts with a")  
    if acc.startswith('b') :  
        print(acc + " starts with b")  
    if acc.startswith('c') :  
        print(acc + " starts with c")  
    if acc.startswith('d') :  
        print(acc + " starts with d")  
    if acc.startswith('e') :  
        print(acc + " starts with e")  
    else :  
        print(acc + ": did not have a starting letter we were looking for:")
```

This is the output

```
ab56 starts with a  
ab56: did not have a starting letter we were looking for:  
eh84 starts with e  
hv76: did not have a starting letter we were looking for:  
ay93 starts with a  
ay93: did not have a starting letter we were looking for:  
zp97: did not have a starting letter we were looking for:  
cd72 starts with c  
cd72: did not have a starting letter we were looking for:
```

That wasnt correct...the `else` was in the wrong place, so maybe

try doing it this way, using two `for` loops

```
accessions = ['ab56', 'eh84', 'hv76', 'ay93', 'zp97', 'cd72']
```

```
wanted_start_letters = 'abcde'
```

Version 1

```
for acc in accessions :  
    for letter in wanted_start_letters :  
        if acc.startswith(letter) :  
            print(letter,acc)  
        else :  
            print('Not what we were looking for:',letter,acc)
```

```
a ab56  
Not a starting letter we were looking for: b ab56  
Not a starting letter we were looking for: c ab56  
Not a starting letter we were looking for: d ab56  
Not a starting letter we were looking for: e ab56  
Not a starting letter we were looking for: a eh84  
Not a starting letter we were looking for: b eh84  
Not a starting letter we were looking for: c eh84  
Not a starting letter we were looking for: d eh84  
e eh84
```

```
Not a starting letter we were looking for: a hv76
Not a starting letter we were looking for: b hv76
Not a starting letter we were looking for: c hv76
Not a starting letter we were looking for: d hv76
Not a starting letter we were looking for: e hv76
a ay93
Not a starting letter we were looking for: b ay93
Not a starting letter we were looking for: c ay93
Not a starting letter we were looking for: d ay93
Not a starting letter we were looking for: e ay93
Not a starting letter we were looking for: a zp97
Not a starting letter we were looking for: b zp97
Not a starting letter we were looking for: c zp97
Not a starting letter we were looking for: d zp97
Not a starting letter we were looking for: e zp97
Not a starting letter we were looking for: a cd72
Not a starting letter we were looking for: b cd72
c cd72
Not a starting letter we were looking for: d cd72
Not a starting letter we were looking for: e cd72
```



We didn't want all of that output!

Version 2

If we find an evaluation that was True, we can "come out of" this `for` loop using `break`

```
for acc in accessions :
    for letter in wanted_start_letters :
        if acc.startswith(letter) :
            print(letter,acc)
            break # exit the loop because it was True and we are finished doing things
        else :
            print('Not a starting letter we were looking for:',letter,acc)
```

```
a ab56
Not a starting letter we were looking for: a eh84
Not a starting letter we were looking for: b eh84
Not a starting letter we were looking for: c eh84
Not a starting letter we were looking for: d eh84
e eh84
Not a starting letter we were looking for: a hv76
Not a starting letter we were looking for: b hv76
Not a starting letter we were looking for: c hv76
Not a starting letter we were looking for: d hv76
Not a starting letter we were looking for: e hv76
a ay93
Not a starting letter we were looking for: a zp97
Not a starting letter we were looking for: b zp97
Not a starting letter we were looking for: c zp97
Not a starting letter we were looking for: d zp97
Not a starting letter we were looking for: e zp97
Not a starting letter we were looking for: a cd72
Not a starting letter we were looking for: b cd72
c cd72
```

That looked better, but we didn't really want all of that output either!

Version 3

If we find an evaluation that was True, come out of this `for` loop using

`break`

We could keep track of the process with a simple variable that will let us know if the letter was ever found

Note the indentation position of the second `if` statement...

```
for acc in accessions:
    for letter in wanted_start_letters :
        status=0
        if acc.startswith(letter) :
            print("The letter",letter,"was found at the beginning of",acc)
            status=1 ; break
    if status==0 :
        print('Did not have a starting letter we were looking for:',acc)
```

```
The letter a was found at the beginning of ab56
The letter e was found at the beginning of eh84
Did not have a starting letter we were looking for: hv76
The letter a was found at the beginning of ay93
Did not have a starting letter we were looking for: zp97
The letter c was found at the beginning of cd72
```

Version 4

Minimal outputs, but include progress indicators...

```
for acc in accessions :
    for letter in wanted_start_letters :
        if acc.startswith(letter) :
            print("\n",letter,acc)
            break
        else :
            print('Not what we were looking for:',letter,acc,end="\x1b[1K\rSearching...")
    if acc == accessions[-1] :
        print("\nAll done now!")
```

```
a ab56
Searching...
e eh84
Searching...
a ay93
Searching...
c cd72
```

All done now!

📌 We can also use the `while` approach to generate a special kind of loop, the `while True` loop that is essentially an "infinite" loop that will continue until it is exited, for example using `break`.

Get information from the user

```
while True :
    ans = input("What is the answer to the ultimate question of life, the universe, and ever")
    if ans == '42' :
        "{} is the correct answer".format(ans)
```

```
break
else :
    print("\tDeep Thought says no...!")
```

What is the answer to the ultimate question of life, the universe, and everything?

Python3

Deep Thought says no...!

What is the answer to the ultimate question of life, the universe, and everything?

42

'42 is the correct answer'

WHAAAAAT?!

[Try this if you want to know why 42 is the correct answer...](#) or maybe it's [this](#)?

📌 Data types

What if you can't remember what you are "allowed" to do with the object you have created?!

Python knows what it can do based on what `type` or class of object it is

This tells us that our `accessions` object is a list

`type(accessions)`

`<class 'list'>`

This tells us things we can do with our variable called `accessions`, which is a list

Python knows it's a list, so...

`help(accessions)`

Help on `list` object:

```
class list(object)
| list() -> new empty list
| list(iterable) -> new list initialized from iterable's items
|
| Methods defined here:
|
| __add__(self, value, /)
|     Return self+value.
|
| __contains__(self, key, /)
|     Return key in self.
|
| __delitem__(self, key, /)
|     Delete self[key].
|
| __eq__(self, value, /)
|     Return self==value.
|
| __ge__(self, value, /)
|     Return self>=value.
|
| __getattr__(self, name, /)
|     Return getattr(self, name).
|
| __getitem__(...)
|     x.__getitem__(y) <==> x[y]
|
| __gt__(self, value, /)
```



```
|         Return self>value.
|
|     __iadd__(self, value, /)
|         Implement self+=value.
|
|     __imul__(self, value, /)
|         Implement self*=value.
|
|     __init__(self, /, *args, **kwargs)
|         Initialize self.  See help(type(self)) for accurate signature.
|
|     __iter__(self, /)
|         Implement iter(self).
|
|     __le__(self, value, /)
|         Return self<=value.
|     __setitem__(self, key, value, /)
|         Set self[key] to value.
|
|     __sizeof__(...)
|         L.__sizeof__() -- size of L in memory, in bytes
|
|     append(...)
|         L.append(object) -> None -- append object to end
|
|     clear(...)
|         L.clear() -> None -- remove all items from L
|
|     copy(...)
|         L.copy() -> list -- a shallow copy of L
|
|     count(...)
|         L.count(value) -> integer -- return number of occurrences of value
|
|     extend(...)
|         L.extend(iterable) -> None -- extend list by appending elements from the iterable
|
|     index(...)
|         L.index(value, [start, [stop]]) -> integer -- return first index of value.
|         Raises ValueError if the value is not present.
|
|     insert(...)
|         L.insert(index, object) -- insert object before index
|
|     pop(...)
|         L.pop([index]) -> item -- remove and return item at index (default last).
|         Raises IndexError if list is empty or index is out of range.
|
|     remove(...)
|         L.remove(value) -> None -- remove first occurrence of value.
|         Raises ValueError if the value is not present.
|
|     reverse(...)
|         L.reverse() -- reverse *IN PLACE*
|
|     sort(...)
|         L.sort(key=None, reverse=False) -> None -- stable sort *IN PLACE*
|
|     -----
|     Data and other attributes defined here:
|
|     __hash__ = None
```

Remember the fun things we can do with lists?

accessions

```
['ab56', 'eh84', 'hv76', 'ay93', 'zp97', 'cd72']
```

```
accessions.append("APPENDED")
```

```
accessions
```

```
['ab56', 'eh84', 'hv76', 'ay93', 'zp97', 'cd72', 'APPENDED']
```

```
accessions.insert( 2 , "INSERTED")
```

```
accessions
```

```
['ab56', 'eh84', 'INSERTED', 'hv76', 'ay93', 'zp97', 'cd72', 'APPENDED']
```

```
accessions.sort()
```

```
accessions
```

```
['APPENDED', 'INSERTED', 'ab56', 'ay93', 'cd72', 'eh84', 'hv76', 'zp97']
```

```
accessions.reverse()
```

```
accessions
```

```
['zp97', 'hv76', 'eh84', 'cd72', 'ay93', 'ab56', 'INSERTED', 'APPENDED']
```

```
accessions.reverse()
```

```
accessions
```

```
['APPENDED', 'INSERTED', 'ab56', 'ay93', 'cd72', 'eh84', 'hv76', 'zp97']
```

```
accessions.remove("APPENDED")
```

```
accessions.remove("INSERTED")
```

```
accessions.remove("INSERTED")
```

```
Traceback (most recent call last):  
  File "<stdin>", line 1, in <module>  
ValueError: list.remove(x): x not in list
```

```
accessions
```

```
['ab56', 'ay93', 'cd72', 'eh84', 'hv76', 'zp97']
```

```
accessions.pop()
```

```
'zp97'
```

```
accessions
```

```
['ab56', 'ay93', 'cd72', 'eh84', 'hv76']
```

accessions.pop(2)
'cd72'
accessions
['ab56', 'ay93', 'eh84', 'hv76']

ID2=accessions.pop(2)
accessions
['ab56', 'ay93', 'hv76']
ID2
'eh84'

📌 Multiple True conditions AND or OR

If more than one condition can be True for a given item, then we could use multiple `if` lines:

Establish a new list

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72']
```

Start the loop

for acc in accessions :

Condition 1: element starts with a

```
if acc.startswith('a') :  
    print("Condition 1 True: " + acc + " starts with a")
```

Condition 2: element ends with 6

```
if acc.endswith('6') :  
    print("Condition 2 True: " + acc + " ends with 6")
```

Combined conditions 1 and 2, both must be True: element starts with a and ends with 6

```
if acc.startswith('a') and acc.endswith('6') :  
    print("Both conditions 1 and 2 are True for " + acc)
```

Combined conditions 1 and 2, either could be True: element starts with a or ends with 6

```
if acc.startswith('a') or acc.endswith('6') :  
    print("At least one of conditions 1 and 2 is True for " + acc)
```

make it an even more complex condition

```
if (acc.startswith('b') or acc.startswith('h')) and acc.endswith('4') :  
    print("Complex condition is True for " + acc)
```

Output contains only things which met the criteria we required

```
Condition 1 True: ab56 starts with a  
Condition 2 True: ab56 ends with 6  
Both conditions 1 and 2 are True for ab56  
At least one of conditions 1 and 2 is True for ab56  
Complex condition is True for bh84  
Condition 2 True: hv76 ends with 6  
At least one of conditions 1 and 2 is True for hv76  
Condition 1 True: ay93 starts with a
```

Give feedback
Ask for help

At least one of conditions 1 and 2 are True for ay93
Condition 1 True: ap97 starts with a
At least one of conditions 1 and 2 are True for ap97

📌 Splitting code up

Complex conditions can be hard to read. If we put parentheses `(` and `)` around the whole thing, then our code can be split over multiple lines in pretty much whatever layout we want!

A nicely structured bit of code

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72']
for acc in accessions :
    if (
        ( acc.startswith('b')
          or acc.startswith('h'))
        and acc.endswith('4')
    ) :
        print("Complex condition is True for " + acc)
```

Complex condition is True for bh84

A nicely structured bit of code #2

```
accessions = ['ab56', 'bh84', 'hv76', 'ay93', 'ap97', 'bd72']
for acc in accessions :
    if (
        ( acc.startswith('b')
          or acc.startswith('h'))
          and acc.endswith('4')
    ) :
        print("Complex condition is True for " + acc)
```

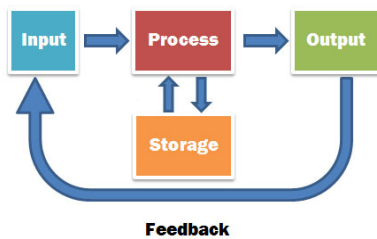
Complex condition is True for bh84

Let's try and be tidy...

Make a directory within your git repository area for today's work, and change directory to it.

```
mkdir ~/Exercises/Lecture14
```

```
cd ~/Exercises/Lecture14
```



1. Processing tabular data

An intro to *Drosophila*



- Copy the text file called `data.csv` from the directory `/localdisk/data/BPSM/Lecture14/` to your area.
- The file contains some made-up data for a number of genes.
- Each line contains the following fields for a single gene in this order: species name, sequence, gene name, expression level.
- The fields are separated by commas (hence the "csv" suffix on the filename: csv stands for **C**omma **S**eparated **V**alues)

You need to write a Python programme/script that can:

1. Print out the gene names for all genes from the species *Drosophila melanogaster* or *Drosophila simulans*.
2. Print out the gene names for all genes that are between 90 and 110 bases long.
3. Print out the gene names for all genes whose AT content is less than 0.5 and whose expression level is greater than 200.
4. Print out the gene names for all genes whose name begins with "k" or "h" except those belonging to *Drosophila melanogaster*.
5. For each gene, print out a message giving the gene name and saying whether its AT content is high (greater than 0.65), low (less than 0.45) or medium (between 0.45 and 0.65).

hint: remember how we used a loop to read the contents of a file line-by-line in the last lecture...

2. 📈K-mer counting

K-mers are short DNA subsequences with length "k" bases, and are usually generated using the "sliding window" method we used in the last lecture's exercise.

Write a programme/script that, given any DNA sequence, will print all the k-mers (e.g. 4-mers) that occur more than some number of times **n** (you chose what the number **n** is!).

For example, with

```
dna="ATGCATCATG"  
k=2 # kmer size  
n=2 # more than this number found
```

Sliding window with offset of 1 and with k=2 gives:

```
AT  
TG  
GC  
CA  
AT ... and so on
```

so the result for this example should be **AT** because the kmers (k) are 2 bases long, and there are 3 instances of AT (n was 2, and 3 is more than 2).

Neither **CA** nor **TG** get listed: they do appear twice each, but 2 is not more than 2....!

3. 📈Pairwise distances: how similar are two sequences?

Here is a list of DNA sequences that are all equal in length, with varying degrees of similarity to each other:

```
['ATTGTACGG', 'AATGAACCG', 'AATGAACCC', 'AATGGGAAT']
```

Write a programme/script that calculates and prints, for each pair of sequences, the percentage of identical positions (e.g. base #4 in seq 1 is the same as base #4 in seq 4, and so on).

Hint:

```
if base_in_seq1 == base_in_seq2:  
    do something
```

[Give feedback](#)

[Ask for help](#)

Possible answers



1. Processing tabular data

Linux file manipulations first, we need to copy the exercise file over!

```
mkdir ~/Exercises/Lecture14
```

```
cd ~/Exercises/Lecture14
```

Copy the provided exercise files into the current directory

```
cp /localdisk/data/BPSM/Lecture14/* .
```

Check that we have what we need

```
ls -l
```

```
data.csv
```

```
head data.csv
```

```
Drosophila melanogaster,atatatatatcgcgatatatacgactatatgcattaattatagcatatcgatatatatatcgata
Drosophila melanogaster,actgtgacgtgtactgtacgactatcgatacgtagtactgatcgctactgtaatgcatccatgctg
Drosophila simulans,atcgatcatgtcgatcgatgatgcacccgactatogtcgatcgatcgatcgatcgatcgatcgatcgat
Drosophila yakuba,cgcgcgctcgcgcatacggcctaatagcgcgcgctagcgatgc,hdt739,85
Drosophila ananassae,ttacgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgat
Drosophila ananassae,gcatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgat
```

```
wc data.csv
```

```
6 12 733 data.csv
```

Equivalent commands in the python3 environment (if we hadnt done it in Linux already)

```
import os, subprocess, shutil
```

```
os.mkdir("/localdisk/home/myuun/Exercises/Lecture14")
```

```
os.chdir("/localdisk/home/myuun/Exercises/Lecture14")
```

```
os.getcwd()
```

```
'/localdisk/home/myuun/Exercises/Lecture14'
```

```
os.listdir()
```

```
[]
```

```
shutil.copy("/localdisk/data/BPSM/Lecture14/data.csv", ".")
```

```
os.listdir()
```

```
['data.csv']
```

```
print(open("data.csv").read())
```

```
Drosophila melanogaster,atatatatatcgcggtatatatacgactatatgcattaattatagcatatcgatatatat  
Drosophila melanogaster,actgtgacgtgtactgtacgactatcgatacgtagtactgatcgctactgtaatgcatc  
Drosophila simulans,atcgatcatgtcgatcgatgatgcacccgactatcgtcgatcgatcgatcgatcgatcgatc  
Drosophila yakuba,cgcgcgctcgcgcatacggcctaattgcgcgcgctagcgatgc,hdt739,85  
Drosophila ananassae,ttacgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcgatcg  
Drosophila ananassae,gcatcgatcgatcgcgggcgatcgatcgcgatcgatcgatcgatcgatcgatcgatcgatcg
```

```
subprocess.call("wc data.csv", shell=True)
```

```
6 12 733 data.csv  
0
```

Could also have a quick look this way

```
open("data.csv").readline()
```

```
'Drosophila
```

```
melanogaster,atatatatatcgcggtatatatacgactatatgcattaattatagcatatcgatatatatatcgata
```

Each line contains the following fields for a single gene in this order: species name, sequence, gene name, expression level.

The fields are separated by commas (hence the "csv" suffix on the filename: csv stands for **C**omma **S**eparated **V**alues)

You need to write a Python programme/script that can:

1. Print out the gene names for all genes from the species *Drosophila melanogaster* or *Drosophila simulans*.
2. Print out the gene names for all genes that are between 90 and 110 bases long.

3. Print out the gene names for all genes whose AT content is less than 0.5 and whose expression level is greater than 200.
4. Print out the gene names for all genes whose name begins with "k" or "h" except those belonging to *Drosophila melanogaster*.
5. For each gene, print out a message giving the gene name and saying whether its AT content is high (greater than 0.65), low (less than 0.45) or medium (between 0.45 and 0.65).

This is my script, saved in the ~/Exercises/Lecture14 directory, called

Exercises_01.py

Programme/script to process the Drosophila gene expression set

Read the file into a file object so that we can process in a `for` loop

Fields are comma separated, open the file connection but we dont need to `read()` it

`flygenedata = open("data.csv")`

`for` loop to read the file line by line

`for geneline in flygenedata :`

Make each line into a list, using `split()` and comma as delimiter

`gene_info = geneline.split(",")`

Define our variables from the `split()`-generated list elements

```
species = gene_info[0]
geneseqs = gene_info[1].upper()
seqlengths = len(gene_info[1])
genenames = gene_info[2]
expressionlevel = int(gene_info[3])
```

AT content calculations, default status is "low"

```
atcontent = (geneseqs.count('A')+geneseqs.count('T'))/seqlengths
atstatus = "low"
if(atcontent >= 0.45 and atcontent <= 0.65) :
    atstatus="medium"
if(atcontent>0.65) :
    atstatus="high"
```

Answer the questions, also including a "condition fail" output for checking purposes

```
if "melanogaster" in species or "simulans" in species:
    print("Question 1 (melanogaster or simulans): "+ species + " "+genenames)
else :
    print("Question 1 (FAIL): "+ species + " "+genenames)
if seqlengths > 90 and seqlengths < 110:
    print("Question 2 (seqlength >90 and <110): "+ species + " "+genenames)
else :
    print("Question 2 (FAIL): "+ species + " "+genenames)
if atcontent < 0.5 and expressionlevel > 200:
    print("Question 3 (AT content <0.5, expr > 200): "+ species + " "+genenames)
else :
    print("Question 3 (FAIL): "+ species + " "+genenames)
if (genenames.startswith("k") or genenames.startswith("h")) and ("Drosophila melanogas
```

```
        print("Question 4 (names start with k or h, not melanogaster): " + species + " " +  
else    :  
        print("Question 4 (FAIL): " + species + " " + genenames)  
print("Question 5 (AT content status): " + genenames + " " + atstatus)
```

Start python

python3 # or p3

Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.

Run my script using `import`: it looks for it locally

import Exercises_01

```
Question 1 (melanogaster or simulans): Drosophila melanogaster kdy647  
Question 2 (seqlength >90 and <110): Drosophila melanogaster kdy647  
Question 3 (FAIL): Drosophila melanogaster kdy647  
Question 4 (FAIL): Drosophila melanogaster kdy647  
Question 5 (AT content status): kdy647 high  
Question 1 (melanogaster or simulans): Drosophila melanogaster jdg766  
Question 2 (FAIL): Drosophila melanogaster jdg766  
Question 3 (FAIL): Drosophila melanogaster jdg766  
Question 4 (FAIL): Drosophila melanogaster jdg766  
Question 5 (AT content status): jdg766 medium  
Question 1 (melanogaster or simulans): Drosophila simulans kdy533  
Question 2 (FAIL): Drosophila simulans kdy533  
Question 3 (FAIL): Drosophila simulans kdy533  
Question 4 (names start with k or h, not melanogaster): Drosophila simulans kdy533  
Question 5 (AT content status): kdy533 medium  
Question 1 (FAIL): Drosophila yakuba hdt739  
Question 2 (FAIL): Drosophila yakuba hdt739  
Question 3 (FAIL): Drosophila yakuba hdt739  
Question 4 (names start with k or h, not melanogaster): Drosophila yakuba hdt739  
Question 5 (AT content status): hdt739 low  
Question 1 (FAIL): Drosophila ananassae hdu045  
Question 2 (FAIL): Drosophila ananassae hdu045  
Question 3 (FAIL): Drosophila ananassae hdu045  
Question 4 (names start with k or h, not melanogaster): Drosophila ananassae hdu045  
Question 5 (AT content status): hdu045 medium  
Question 1 (FAIL): Drosophila ananassae teg436  
Question 2 (seqlength >90 and <110): Drosophila ananassae teg436  
Question 3 (AT content <0.5, expr > 200): Drosophila ananassae teg436  
Question 4 (FAIL): Drosophila ananassae teg436  
Question 5 (AT content status): teg436 medium
```

What happens if you try it again?

import Exercises_01

Why might that be?! It's all about `__name__` and `__main__` ... see [this page](#) or [this page](#) for some good examples and explanations!

Once you were happy with how the script was working, you could:

- remove the print "FAIL" lines?

- maybe save the outputs to files, one for each of the questions?

etc., etc

2. Kmer counting

K-mers are short DNA sub-sequences with length "k" bases, and are usually generated using the sliding window method we used in the last lecture's exercise. Write a programme/script that, given any DNA sequence, will print all the k-mers (e.g. 4-mers) that occur more than some number (n) of times (you chose what that number is).

For example, with

```
dna="ATGCATCATG"  
k=2  
n=2
```

the output should be **AT** because the kmers (k) are 2 bases long, and there are 3 instances of AT (n was 2, and 3 is more than 2).

Neither **CA** nor **TG** get listed: they do appear twice each, but 2 is not more than 2....!

We don't have much in the way of defined goals for this question!

We need something generic that we could maybe plug numbers in as and when needed.

This is essentially where having a function comes in... more on that in the next lecture!

This is my script, saved in the ~/Exercises/Lecture14 directory, called

Exercises_02.py

Write a programme/script that, given any DNA sequence, will print all the k-mers (e.g. 4-mers) that occur more than n times.

Written by s123456 on 31 Oct 2025

-----

INPUT a made-up test sequence, assume it's good and all the same (lower) case

```
sequencein =  
"atatatatatcgcggtatatatacgactatatgcattaattatagcatatcgatatatatcgatattatcgattatcgcgcgtaattatc"
```

Possible_kmer_sizes `list(range(2,int(len(sequencein)-1)))` = 2 to 96

Arbitrarily choose a smaller set (2-6) for now!

```
possible_kmer_sizes = list(range(2,7))
```

Set the minimum frequency

```
kmerfound_minimum = 3
```

PROCESS using a `for` loop for the kmer sizes

```
for window in possible_kmer_sizes :  
    kmersfound = []  
    kmerrange = list(range(0,len(sequencein)))
```

Also use a `for` loop for getting the kmer sequences

```
    for startingbase in kmerrange:  
        if (startingbase+window)<len(sequencein)+1 :  
            seqout = (sequencein)[startingbase:startingbase+window]  
            kmersfound = kmersfound + [seqout]
```

OUTPUT the frequencies of the non-redundant set of kmer sequences

```
    nonredundantset = list(set(kmersfound))  
    for kmerfrequencies in nonredundantset :  
        if kmersfound.count(kmerfrequencies) > kmerfound_minimum :  
            print("Lots! " + str(kmerfrequencies)+" "+str(kmersfound.count(kmerfrequencies))  
        else :  
            print(str(kmerfrequencies) + " " + str(kmersfound.count(kmerfrequencies)))
```

Start python

```
python3 # or p3
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.
```

Run my script using `import`: it looks for it locally

```
import Exercises_02
```

These are (some of) the outputs for THIS sequence (yours WILL be different!)

```
Kmer: 2: found: 96 kmers, nr set of 14.  
ga 3  
aa 2  
ag 1  
ct 1  
ac 3  
ca 3  
Lots! gc 6  
Lots! at 29  
Lots! tc 5  
Lots! ta 26  
Lots! cg 9  
gt 2  
tg 1  
Lots! tt 5  
Kmer: 3: found: 95 kmers, nr set of 25.  
gca 3  
atg 1  
agc 1
```

```
acg 2
taa 2
gcg 3
Lots! tta 5
cat 3
Lots! tat 21
gat 2
cta 1
gac 1
Lots! atc 5
Lots! cgc 4
cga 3
Lots! att 5
Lots! ata 18
Lots! tcg 4
cgt 2
aat 2
gta 2
tag 1
tgc 1
act 1
tac 2
Kmer: 4: found: 94 kmers, nr set of 38.
... etc.
```

Code works, but was the question/biology sensible?!

How might you get the user to enter the sequence from the keyboard?

Might this be a suitable approach!?

```
sequencein = input("Please enter the raw DNA sequence you want to
analyse\n").upper()
```

Please enter the raw DNA sequence you want to analyse

asdksljdkjasldkjalskjdakjd

```
print("Thanks for entering the sequence",sequencein,"(which
is",len(sequencein),"characters long)")
```

Thanks for entering the sequence ASDKSLDKJASLDKJALSKJDLAKJD (which is 26 characters long)

....

....

Situations such as these, where we do the same set of processing steps many times, but with different inputs, are best dealt with using "generic code", and in this case, the generic code will be a **function**: our next topic!

3. Pairwise distances: how similar are two sequences?

Here is a list of DNA sequences that are all equal in length, with varying degrees

of similarity:

```
['ATTGTACGG', 'AATGAACCG', 'AATGAACCC', 'AATGGGAAT']
```

Write a programme/script that calculates and prints, for each pair of sequences, the percentage of identical positions.

Hint:

```
if base_in_seq1 == base_in_seq2 :  
    do something
```

This is my script, saved in the ~/Exercises/Lecture14 directory, called
`Exercises_03.py`

```
# programme/script to compare a small set of 9-base sequences, pairwise  
# determining a distance score based on percent identity  
# All sequences are of equal length (pew, that's good!)  
# Written by s123456 on 31 Oct 2025  
# -----  
#
```

```
# INPUT
```

```
input_seqs = ['ATTGTACGG', 'AATGAACCG', 'AATGAACCC', 'AATGGGAAT']
```

```
# PROCESS BIT
```

```
for_range = list(range(0,len(input_seqs)))
```

```
for_range
```

```
[0, 1, 2, 3]
```

```
# Need loop for x and y accessions of pairwise comparisons, just do all of  
# them!
```

```
# "X axis" loop
```

```
for Xaxis_item in for_range:
```

```
# Convert the first element of input_seqs into a list itself
```

```
    first_query = list(input_seqs[Xaxis_item])
```

```
# "Y axis" loop
```

```
    for Yaxis_item in for_range:
```

```
# Reset distance measure
```

```
distance = 0
other_query = list(input_seqs[Yaxis_item])
```

**# Go along each base in each of the two sequences to
compare if identical or not; score +1 if it is**

```
for base in list(range(0,len(first_query))) :
    print("Index" + str(base) + ":" + ## this one
          str(first_query[base]) + "," + str(other_query[base]) + "...")
    if first_query[base] == other_query[base] :
        distance = distance + 1
```

OUTPUT pairwise comparison done, output details

DON'T include the self-comparisons in the summary output

```
if input_seqs[Xaxis_item] != input_seqs[Yaxis_item] :
```

Scores

```
print(str(distance) + " identities between " + input_seqs[Xaxis_item] +
      " and " + input_seqs[Yaxis_item])
```

Percentages

```
print("\t" + str(int(100*(distance/len(input_seqs[Xaxis_item])))) +
      " percent similarity between " + input_seqs[Xaxis_item] + " and " + input_s
```

Start python

python3 # or p3

Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.

Run my script using `import` : it looks for it locally

import Exercises_03

Output lines are:

```
Index0:A,A...
Index1:T,T...
Index2:T,T...
Index3:G,G...
Index4:T,T...
Index5:A,A...
Index6:C,C...
Index7:G,G...
Index8:G,G...
Index0:A,A...
Index1:T,A...
Index2:T,T...
Index3:G,G...
Index4:T,A...
Index5:A,A...
Index6:C,C...
Index7:G,C...
Index8:G,G...
6 identities between ATTGTACGG and AATGAACCG
    66 percent similarity between ATTGTACGG and AATGAACCG
Index0:A,A...
Index1:T,A...
... etc
```

**# If we disable the base_vs_base compare print line
by adding a comment # to the front of the print line,
the output lines become**

```
6 identities between ATTGTACGG and AATGAACCG
  66 percent similarity between ATTGTACGG and AATGAACCG
5 identities between ATTGTACGG and AATGAACCC
  55 percent similarity between ATTGTACGG and AATGAACCC
3 identities between ATTGTACGG and AATGGGAAT
  33 percent similarity between ATTGTACGG and AATGGGAAT
6 identities between AATGAACCG and ATTGTACGG
  66 percent similarity between AATGAACCG and ATTGTACGG
8 identities between AATGAACCG and AATGAACCC
  88 percent similarity between AATGAACCG and AATGAACCC
4 identities between AATGAACCG and AATGGGAAT
  44 percent similarity between AATGAACCG and AATGGGAAT
5 identities between AATGAACCC and ATTGTACGG
  55 percent similarity between AATGAACCC and ATTGTACGG
8 identities between AATGAACCC and AATGAACCG
  88 percent similarity between AATGAACCC and AATGAACCG
4 identities between AATGAACCC and AATGGGAAT
  44 percent similarity between AATGAACCC and AATGGGAAT
3 identities between AATGGGAAT and ATTGTACGG
  33 percent similarity between AATGGGAAT and ATTGTACGG
4 identities between AATGGGAAT and AATGAACCG
  44 percent similarity between AATGGGAAT and AATGAACCG
4 identities between AATGGGAAT and AATGAACCC
  44 percent similarity between AATGGGAAT and AATGAACCC
```

exit()

There IS an alternative way to answer this, using [numpy arrays](#)... which is at the end of this page, something for a later time?!

📈git and GitHub time!

ls -l

```
data.csv
Exercises_01.py
Exercises_02.py
Exercises_03.py
__pycache__
```

git st

```
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
```

```
(use "git restore <file>..." to discard changes in working directory)
deleted:    ../Lecture04/CodeFiles.tar.gz
modified:   ../Lecture06/run_this_script_v1
```

Untracked files:

```
(use "git add <file>..." to include in what will be committed)
```

```
../BN.out
../BN.sh
../Lecture04/Als_ICA/
../Lecture04/all_the_things_I_did
../Lecture04/outs/
../Lecture05/Country.Mozambique.details
../Lecture07/Cosmoscarta.nuc.fa
../Lecture07/Cosmoscarta.nuc.gis
../Lecture07/myfileofaccessions.txt
../Lecture12/AJ223353.fasta
../Lecture12/AJ223353.genbank
../Lecture12/AJ223353_noheader.fasta
../Lecture12/AJ223353_noheader.fasta2
../Lecture12/AJ223353_noheader.fasta3
../Lecture12/All_exons.fasta
../Lecture12/All_noncodings.fasta
../Lecture12/local_exon01.fasta
../Lecture12/local_exon02.fasta
../Lecture12/local_intron01.fasta
../Lecture12/my_dna.txt
../Lecture12/plain_genomic_seq.txt
../Lecture12/quick_blast_check.out
../Lecture12/remote_exon01.fasta
../Lecture12/remote_noncoding01.fasta
../Lecture12/remote_noncoding02.fasta
../Lecture12/shutilversion.txt
../Lecture13/100_199/
../Lecture13/200_299/
../Lecture13/300_399/
../Lecture13/400_499/
../Lecture13/500_599/
../Lecture13/600_699/
../Lecture13/700_799/
../Lecture13/800_899/
../Lecture13/900_999/
../Lecture13/Cleaned_sequences.txt
../Lecture13/Cleaned_sequences2.txt
../Lecture13/Cleaned_sequences3.txt
../Lecture13/Exercise2_coding_sequence.fasta
../Lecture13/Lecture13.tar.gz
../Lecture13/dna_files/
../Lecture13/exons.txt
../Lecture13/genomic_dna2.txt
../Lecture13/input.txt
../Lecture13/remote_exon01.fasta
./
```

no changes added to commit (use "git add" and/or "git commit -a")

You might want to add today's files?

git add *.py

git commit -m "My scripts from the python conditionals exercises Py04"

```
[master 3b8bf41] My scripts from the python conditionals exercises Py04
3 files changed, 113 insertions(+)
create mode 100644 Lecture14/Exercises_01.py
create mode 100644 Lecture14/Exercises_02.py
create mode 100644 Lecture14/Exercises_03.py
```

Which link to GitHub did we want to use for pushing?

git remote

```
AllLectures
Lecture07
origin
```

Push our local repository to the one we just created at GitHub

git push AllLectures master

```
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 256 threads
Compressing objects: 100% (6/6), done.
Writing objects: 100% (6/6), 2.51 KiB | 1.25 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/B123456-2121/BPSM_LectureStuff.git
 8f58f4c..3b8bf41  master -> master
```

git log

```
3b8bf41 (HEAD -> master, AllLectures/master) My scripts from the python conditionals exerc
8f58f4c Scripts from the python lists and loops lecture 3
e444ef8 Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fbe5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

All done!

How else might this have been done?

Have a look below to see an example using the `numpy` module...

"The fundamental package for scientific computing with Python", it has it's own website <https://numpy.org/>

```
# Make our input_seqs into one big long string, then compare all bases  
against all bases
```

```
input_seqs
```

```
['ATTGTACGG', 'AATGAACCG', 'AATGAACCC', 'AATGGGAAT']
```

```
s=""
```

```
myseqs=s.join(str(e) for e in input_seqs)
```

```
myseqs
```

```
'ATTGTACGGAATGAACCGAATGAACCCAATGGGAAT'
```

```
# Generate an array "table" to put values in: a 2 dimensional "numpy" array
```

```
# To do this we need the numpy module (numerical python)
```

```
# For details about this huge module, see http://www.numpy.org/
```

```
# We can either import the module as it is, or create a short-form alias for it
```

```
import numpy
```

```
# OR
```

```
import numpy as np
```

```
# What can np do? np DOT TAB TAB
```

```
np.
```

```
Display all 556 possibilities? (y or n)
numpy.ALLOW_THREADS          numpy.copyto(
numpy.BUF_SIZE               numpy.corrcoeff(
numpy.CLIP                   numpy.correlate(
numpy.DataSource(           numpy.cos(
numpy.ERR_CALL               numpy.cosh(
numpy.ERR_DEFAULT           numpy.count_nonzero(
numpy.ERR_IGNORE            numpy.cov(
numpy.ERR_LOG               numpy.cross(
numpy.ERR_PRINT             numpy.csingle(
numpy.ERR_RAISE             numpy.ctypeslib
numpy.ERR_WARN              numpy.cumprod(
numpy.FLOATING_POINT_SUPPORT numpy.cumproduct(
numpy.FPE_DIVIDEBYZERO      numpy.cumsum(
numpy.FPE_INVALID           numpy.datetime64(
numpy.FPE_OVERFLOW          numpy.datetime_as_string(
numpy.FPE_UNDERFLOW         numpy.datetime_data(
numpy.False_                numpy.deg2rad(
numpy.Inf                   numpy.degrees(
numpy.Infinity              numpy.delete(
numpy.MAXDIMS               numpy.deprecate(
numpy.MAY_SHARE_BOUNDS      numpy.deprecate_with_doc(
numpy.MAY_SHARE_EXACT       numpy.diag(
numpy.NaN                   numpy.diag_indices(
numpy.NINF                  numpy.diag_indices_from(
numpy.intersect1d(
numpy.intp(
numpy.invert(
numpy.is_busday(
numpy.isclose(
numpy.iscomplex(
numpy.iscomplexobj(
numpy.isfinite(
numpy.isfortran(
numpy.isin(
numpy.isinf(
numpy.isnan(
numpy.isnat(
numpy.isneginf(
numpy.isposinf(
numpy.isreal(
numpy.isrealobj(
numpy.isscalar(
numpy.issctype(
numpy.issubclass_(
numpy.issubdtype(
numpy.issubsttype(
numpy.iterable(
numpy.ix_(
numpy.ptp(
numpy.put(
numpy.put_along_axis(
numpy.putmask(
numpy.quantile(
numpy.r_
numpy.rad2deg(
numpy.radians(
numpy.random
numpy.ravel(
numpy.ravel_multi_index(
numpy.real(
numpy.real_if_close(
numpy.rec
numpy.reccarray(
numpy.recfromcsv(
numpy.recfromtxt(
numpy.reciprocal(
numpy.record(
numpy.remainder(
numpy.repeat(
numpy.require(
numpy.reshape(
numpy.resize(
```


numpy.NZERO	numpy.diagflat()	numpy.kaiser()	numpy.result_type()
numpy.NaN	numpy.diagonal()	numpy.kernel_version	numpy.right_shift()
numpy.PINF	numpy.diff()	numpy.kron()	numpy rint()
numpy.PZERO	numpy.digitize()	numpy.lcm()	numpy.roll()
numpy.RAISE	numpy.disp()	numpy.ldexp()	numpy.rollaxis()
numpy.RankWarning()	numpy.divide()	numpy.left_shift()	numpy.roots()
numpy.SHIFT_DIVIDEBYZERO	numpy.divmod()	numpy.less()	numpy.rot90()
numpy.SHIFT_INVALID	numpy.dot()	numpy.less_equal()	numpy.round()
numpy.SHIFT_OVERFLOW	numpy.double()	numpy.lexsort()	numpy.round_()
numpy.SHIFT_UNDERFLOW	numpy.dsplit()	numpy.lib	numpy.row_stack()
numpy.ScalarType	numpy.dstack()	numpy.linalg	numpy.s_
numpy.True_	numpy.dtype	numpy.linspace()	numpy.safe_eval()
numpy.UFUNC_BUFSIZE_DEFAULT	numpy.dtypes	numpy.little_endian	numpy.save()
numpy.UFUNC_FYVALS_NAME	numpy.e	numpy.load()	numpy.savetxt()
numpy.WRAP	numpy.ediff1d()	numpy.loadtxt()	numpy.savez()
numpy.abs()	numpy.einsum()	numpy.log()	numpy.savez_compressed()
numpy.absolute()	numpy.einsum_path()	numpy.log10()	numpy.sctype2char()
numpy.add()	numpy.emath	numpy.log1p()	numpy.sctypeDict
numpy.add_docstring()	numpy.empty()	numpy.log2()	numpy.sctypes
numpy.add_newdoc()	numpy.empty_like()	numpy.logaddexp()	numpy.searchsorted()
numpy.add_newdoc_ufunc()	numpy.equal()	numpy.logaddexp2()	numpy.select()
numpy.all()	numpy.errstate()	numpy.logical_and()	numpy.set_numeric_ops()
numpy.allclose()	numpy.euler_gamma	numpy.logical_not()	numpy.set_printoptions()
numpy.alltrue()	numpy.exceptions	numpy.logical_or()	numpy.set_string_function()
numpy.amax()	numpy.exp()	numpy.logical_xor()	numpy.setbufsize()
numpy.amin()	numpy.exp2()	numpy.logspace()	numpy.setdiff1d()
numpy.angle()	numpy.expand_dims()	numpy.longcomplex()	numpy.seterr()
numpy.any()	numpy.explm()	numpy.longdouble()	numpy.seterrcall()
numpy.append()	numpy.extract()	numpy.longfloat()	numpy.seterrobj()
numpy.apply_along_axis()	numpy.eye()	numpy.longlong()	numpy.setxor1d()
numpy.apply_over_axes()	numpy.fabs()	numpy.lookfor()	numpy.shape()
numpy.arange()	numpy.fastCopyAndTranspose()	numpy.ma	numpy.shares_memory()
numpy.arccos()	numpy.fft	numpy.mask_indices()	numpy.short_
numpy.arccosh()	numpy.fill_diagonal()	numpy.mat()	numpy.show_config()
numpy.arcsin()	numpy.find_common_type()	numpy.matmul()	numpy.show_runtime()
numpy.arcsinh()	numpy.finfo()	numpy.matrix()	numpy.sign()
numpy.arctan()	numpy.fix()	numpy.max()	numpy.signbit()
numpy.arctan2()	numpy.flatiter()	numpy.maximum()	numpy.signedinteger()
numpy.arctanh()	numpy.flatnonzero()	numpy.maximum_sctype()	numpy.sin()
numpy.argmax()	numpy.flexible()	numpy.may_share_memory()	numpy.sinc()
numpy.argmin()	numpy.flip()	numpy.mean()	numpy.single()
numpy.argsortpartition()	numpy.fliplr()	numpy.median()	numpy.singlecomplex()
numpy.argsort()	numpy.flipud()	numpy.memmap()	numpy.sinh()
numpy.argwhere()	numpy.float128()	numpy.meshgrid()	numpy.size()
numpy.around()	numpy.float16()	numpy.mgrid()	numpy.sometrue()
numpy.array()	numpy.float32()	numpy.min()	numpy.sort()
numpy.array2string()	numpy.float64()	numpy.min_scalar_type()	numpy.sort_complex()
numpy.array_equal()	numpy.float_()	numpy.minimum()	numpy.source()
numpy.array_equiv()	numpy.float_power()	numpy.mintypecode()	numpy.spacing()
numpy.array_repr()	numpy.floating()	numpy.mod()	numpy.split()
numpy.array_split()	numpy.floor()	numpy.modf()	numpy.sqrt()
numpy.array_str()	numpy.floor_divide()	numpy.moveaxis()	numpy.square()
numpy.asanyarray()	numpy.fmax()	numpy.msort()	numpy.squeeze()
numpy.asarray()	numpy.fmin()	numpy.multiply()	numpy.stack()
numpy.asarray_chkfinite()	numpy.fmod()	numpy.nan	numpy.std()
numpy.ascontiguousarray()	numpy.format_float_positional()	numpy.nan_to_num()	numpy.str_
numpy.asfarray()	numpy.format_float_scientific()	numpy.nanargmax()	numpy.string_()
numpy.asfortranarray()	numpy.format_parser()	numpy.nanargmin()	numpy.subtract()
numpy.asmatrix()	numpy.frexp()	numpy.nancumprod()	numpy.sum()
numpy.atleast_1d()	numpy.from_dlpack()	numpy.nancumsun()	numpy.swapaxes()
numpy.atleast_2d()	numpy.frombuffer()	numpy.nanmax()	numpy.take()
numpy.atleast_3d()	numpy.fromfile()	numpy.nanmean()	numpy.take_along_axis()
numpy.average()	numpy.fromfunction()	numpy.nanmedian()	numpy.tan()
numpy.bartlett()	numpy.fromiter()	numpy.nanmin()	numpy.tanh()
numpy.base_repr()	numpy.frompyfunc()	numpy.nanpercentile()	numpy.tensordot()
numpy.binary_repr()	numpy.fromregex()	numpy.nanprod()	numpy.test()
numpy.bincount()	numpy.fromstring()	numpy.nanquantile()	numpy.testing
numpy.bitwise_and()	numpy.full()	numpy.nanstd()	numpy.tile()
numpy.bitwise_not()	numpy.full_like()	numpy.nansum()	numpy.timedelta64()
numpy.bitwise_or()	numpy.gcd()	numpy.nanvar()	numpy.trace()
numpy.bitwise_xor()	numpy.generic()	numpy.nbytes	numpy.tracemalloc_domain
numpy.blackman()	numpy.genfromtxt()	numpy.ndarray	numpy.transpose()
numpy.block()	numpy.geomspace()	numpy.ndenumerate()	numpy.trapz()
numpy.bmat()	numpy.get_array_wrap()	numpy.ndim	numpy.tri()
numpy.bool_()	numpy.get_include()	numpy.ndindex()	numpy.tril()
numpy.broadcast()	numpy.get_printoptions()	numpy.nditer()	numpy.tril_indices()
numpy.broadcast_arrays()	numpy.getbufsize()	numpy.negative()	numpy.tril_indices_from()
numpy.broadcast_shapes()	numpy.geterr()	numpy.nested_iters()	numpy.trim_zeros()
numpy.broadcast_to()	numpy.geterrcall()	numpy.newaxis	numpy.triu()
numpy.busday_count()	numpy.geterrobj()	numpy.nextafter()	numpy.triu_indices()
numpy.busday_offset()	numpy.gradient()	numpy.nonzero()	numpy.triu_indices_from()
numpy.busdaycalendar()	numpy.greater()	numpy.not_equal()	numpy.true_divide()
numpy.byte()	numpy.greater_equal()	numpy.numarray	numpy.trunc()
numpy.byte_bounds()	numpy.half()	numpy.number()	numpy.typecodes
numpy.bytes_()	numpy.hamming()	numpy.obj2sctype()	numpy.type_name()
numpy.c	numpy.hanning()	numpy.object_()	numpy.ubyte()
numpy.can_cast()	numpy.heaviside()	numpy.ogrid	numpy.ufunc()
numpy.cast	numpy.histogram()	numpy.oldnumeric	numpy.uint()
numpy.cbrt()	numpy.histogram2d()	numpy.ones()	numpy.uint16()
numpy.cdoube()	numpy.histogram_bin_edges()	numpy.ones_like()	numpy.uint32()
numpy.ceil()	numpy.histogramdd()	numpy.outer()	numpy.uint64()
numpy.cfloat()	numpy.hsplit()	numpy.packbits()	numpy.uint8()
numpy.char	numpy.hstack()	numpy.pad()	numpy.uintc()
numpy.character()	numpy.hypot()	numpy.partition()	numpy.uintp()
numpy.chararray	numpy.i0()	numpy.percentile()	numpy.ulonglong()
numpy.choose()	numpy.identity()	numpy.pi	numpy.unicode_()
numpy.clip()	numpy.iinfo()	numpy.piecwise()	numpy.union1d()
numpy.clongdouble()	numpy.imag()	numpy.place()	numpy.unique()
numpy.clongfloat()	numpy.in1d()	numpy.poly()	numpy.unpackbits()
numpy.column_stack()	numpy.index_exp	numpy.polyld()	numpy.unravel_index()
numpy.common_type()	numpy.indices()	numpy.polyadd()	numpy.unsignedinteger()
numpy.compare_chararrays()	numpy.inexact()	numpy.polyder()	numpy.unwrap()
numpy.compat	numpy.inf	numpy.polydiv()	numpy.ushort()
numpy.complex128()	numpy.info()	numpy.polyfit()	numpy.vander()
numpy.complex256()	numpy.infty	numpy.polyint()	numpy.var()
numpy.complex64()	numpy.inner()	numpy.polymul()	numpy.vdot()
numpy.complex_()	numpy.insert()	numpy.polynomial	numpy.vectorize()
numpy.complexfloating()	numpy.int16()	numpy.polysub()	numpy.version
numpy.compress	numpy.int32()	numpy.polyval()	numpy.void()
numpy.concatenate()	numpy.int64()	numpy.positive()	numpy.vsplit()
numpy.conj()	numpy.int8()	numpy.power()	numpy.vstack()
numpy.conjugate()	numpy.int_()	numpy.printoptions()	numpy.where()
numpy.convolve()	numpy.intc()	numpy.prod()	

We need a 2-dimensional array, which is called a "rank 2" array
Create a rank 2 array with 2 rows and three columns, using numpy or np

Both commands are the same, they are just creating arrays with different names

```
o = numpy.array([[1,2,3],[4,5,6]])
```

```
a = np.array([[1,2,3],[4,5,6]])
```

What shape is the array? This gives us the number of rows, columns.

```
print(a.shape)
```

```
(2, 3)
```

To output items

```
a
```

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

```
o
```

```
array([[1, 2, 3],  
       [4, 5, 6]])
```

To output items using print()

```
print(a)
```

```
[[1 2 3]  
 [4 5 6]]
```

Create a 12 by 4 array of all zeros

```
b = np.zeros((12,4))
```

```
print(b)
```

```
[[ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]  
 [ 0.  0.  0.  0.]
```

Create a 5 by 2 array of all ones

```
c = np.ones((5,2))
```

```
print(c)
```

```
[[ 1.  1.]  
 [ 1.  1.]  
 [ 1.  1.]  
 [ 1.  1.]  
 [ 1.  1.]]
```

```
# Create a 3 by 2 array with all values being 6
```

```
d = np.full((3,2), 6)
```

```
print(d)
```

```
[[ 6.  6.]  
 [ 6.  6.]  
 [ 6.  6.]]
```

```
# Create a 2 by 2 array filled with random (floating point) values
```

```
e = np.random.random((2,2))
```

```
print(e)
```

```
[[ 0.76920673  0.79515976]  
 [ 0.19348798  0.02439216]]
```

```
# Indexes start at zero, remember!
```

```
e[1,1]
```

```
0.02439216
```

```
# Create a 3 by 3 identity matrix
```

```
f = np.eye(3)
```

```
print(f)
```

```
[[ 1.  0.  0.]  
 [ 0.  1.  0.]  
 [ 0.  0.  1.]]
```

```
# OK, let's use an array for our sequence similarity!
```

```
myseqs
```

```
'ATTGTACGGAATGAACCGAATGAACCCAATGGGAAT'
```

```
len(myseqs)
```

```
36
```

```
# Let's use an identity matrix...
```

```
# eye stands for the i in "identity" ?!
```

```
help(np.eye)
```

```
Help on function eye in module numpy:
```

```
eye(N, M=None, k=0, dtype=<class 'float'>, order='C')  
    Return a 2-D array with ones on the diagonal and zeros elsewhere.
```

Parameters

N : int

Number of rows in the output.

M : int, optional

Number of columns in the output. If None, defaults to `N`.

k : int, optional

Index of the diagonal: 0 (the default) refers to the main diagonal, a positive value refers to an upper diagonal, and a negative value to a lower diagonal.

dtype : data-type, optional

Data-type of the returned array.

order : {'C', 'F'}, optional

Whether the output should be stored in row-major (C-style) or column-major (Fortran-style) order in memory.

.. versionadded:: 1.20.0

Returns

I : ndarray of shape (N,M)

An array where all elements are equal to zero, except for the `k`-th diagonal, whose values are equal to one.

See Also

identity : (almost) equivalent function

diag : diagonal 2-D array from a 1-D array specified by the user.

Examples

```
>>> np.eye(2, dtype=int)
array([[1, 0],
       [0, 1]])
>>> np.eye(3, k=1)
array([[0.,  1.,  0.],
       [0.,  0.,  1.],
       [0.,  0.,  0.]])
```

```
IDmatrix = np.eye(len(myseqs),dtype= int)
```

```
# Now all we need is two for loops to iterate over the "X-axis" and "Y-axis"
```

```
# We will treat the rows as X, and columns as Y
```

```
# We will be doing 36 x 36 (1296) base-to-base comparisons!
```

```
# A "human" counter for the x base position
```

```
xcounter=0
```

```
# The for loops
```

```
for xbase in myseqs :
    xcounter+=1
    ycounter=0
    for ybase in myseqs :
        ycounter+=1
        print(xcounter,ycounter,xbase,ybase)
        if ybase==xbase :
```

```
IDmatrix[(xcounter-1),(ycounter-1)]=1
```

```
1 1 A A
1 2 A T
1 3 A T
1 4 A G
1 5 A T
1 6 A A
1 7 A C
1 8 A G
1 9 A G
1 10 A A
1 11 A A
1 12 A T
1 13 A G
1 14 A A
1 15 A A
```

```
... lots more lines
```

```
36 23 T A
36 24 T A
36 25 T C
36 26 T C
36 27 T C
36 28 T A
36 29 T A
36 30 T T
36 31 T G
36 32 T G
36 33 T G
36 34 T A
36 35 T A
36 36 T T
```

Print out a bit of the array we have populated (rows, columns)

First row only

```
len(IDmatrix[0,])
```

```
36
```

```
IDmatrix[0,]
```

```
array([1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0,
       1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0])
```

First row only, which ones were identical to the first base of `myseqs`

```
IDmatrix[0,]>0
```

```
array([ True, False, False, False, False,  True, False, False, False,
        True,  True, False, False,  True,  True, False, False, False,
        True,  True, False, False,  True,  True, False, False, False,
        True,  True, False, False, False, False,  True,  True, False], dtype=bool)
```

I have put on the actual sequence bases, first 9 compared to first 9

```
myseqs[0:9]
```

```
'ATTGTACGG'
```

```
print(IDmatrix[0:9,0:9])
```

```
  A T T G T A C G G
[[1 0 0 0 0 1 0 0 0]
 [0 1 1 0 1 0 0 0 0]]
```

```
[0 1 1 0 1 0 0 0 0]
[0 0 0 1 0 0 0 1 1]
[0 1 1 0 1 0 0 0 0]
[1 0 0 0 0 1 0 0 0]
[0 0 0 0 0 0 1 0 0]
[0 0 0 1 0 0 0 1 1]
[0 0 0 1 0 0 0 1 1]]
```

Same again, this time with first 18

myseqs[0:18]

'ATTGTACGGAATGAACCG'

print(IDmatrix[0:18,0:18])

```
      A T T G T A C G G | A A T G A A C C G
A [[1 0 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 0]
T [0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0]
T [0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0]
G [0 0 0 1 0 0 0 1 1 0 0 0 1 0 0 0 0 1]
T [0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0]
A [1 0 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 0]
C [0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0]
G [0 0 0 1 0 0 0 1 1 0 0 0 1 0 0 0 0 1]
G [0 0 0 1 0 0 0 1 1 0 0 0 1 0 0 0 0 1]
A [1 0 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 0]
A [1 0 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 0]
T [0 1 1 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0]
G [0 0 0 1 0 0 0 1 1 0 0 0 1 0 0 0 0 1]
A [1 0 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 0]
A [1 0 0 0 0 1 0 0 0 1 1 0 0 1 1 0 0 0]
C [0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0]
C [0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0]
G [0 0 0 1 0 0 0 1 1 0 0 0 1 0 0 0 0 1]]
```

We want the "equivalent position" values, which is the diagonal

IDmatrix[0:9,9:18].diagonal()

array([1, 0, 1, 1, 0, 1, 1, 0, 1])

It will only do the actual diagonal bit!

IDmatrix[0:9,9:19].diagonal()

array([1, 0, 1, 1, 0, 1, 1, 0, 1])

We can process the values in the diagonal

IDmatrix[0:9,9:18].diagonal().sum()

6

Make it a percentage

**print("The similarity
was",int((IDmatrix[0:9,9:18].diagonal().sum()/9)*100),"percent")**

The similarity was 66 percent

and so on for each original sequence

```
# We can do other maths too
```

```
IDmatrix.sum()
```

```
# Sum of the whole array
```

```
362
```

```
# Sum of the columns
```

```
IDmatrix.sum(axis=0)
```

```
array([14, 7, 7, 9, 7, 14, 6, 9, 9, 14, 14, 7, 9, 14, 14, 6, 6, 9, 14, 14, 7, 9, 14, 14,
        6, 6, 6, 14, 14, 7, 9, 9, 9, 14, 14, 7])
```

```
# Sum of the rows (these will be the same values for our data)
```

```
IDmatrix.sum(axis=1)
```

```
array([14, 7, 7, 9, 7, 14, 6, 9, 9, 14, 14, 7, 9, 14, 14, 6, 6, 9, 14, 14, 7, 9, 14, 14,
        6, 6, 6, 14, 14, 7, 9, 9, 9, 14, 14, 7])
```

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!



Sources used